

1. b. Unrolled List
–Insert Element
–Time Complexity Analysis

Program:

```
#define MAX_ELEMENTS 4

// Structure for the Unrolled Linked List Node
struct Node {
    int numElements;
    int elements[MAX_ELEMENTS];
    struct Node* next;
};

// Global head pointer
struct Node* head = NULL;

void insertElement(int val) {
    if (head == NULL) {
        head = (struct Node*)malloc(sizeof(struct Node));
        head->numElements = 1;
        head->elements[0] = val;
        head->next = NULL;
        cout << "Inserted " << val << " into a new head node.\n";
        return;
    }
    struct Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    if (temp->numElements < MAX_ELEMENTS) {
        temp->elements[temp->numElements] = val;
        temp->numElements++;
        cout << "Inserted " << val << " into the existing last
node.\n";
    }
}
```

```

else {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->numElements = 1;
    newNode->elements[0] = val;
    newNode->next = NULL;

    temp->next = newNode;
    cout << "Node full. Inserted " << val << " into a newly
created node.\n";
}
}
...

case 1:
    cout << "Enter integer value to insert: ";
    cin >> value;
    insertElement(value);
    break;

```

Time Complexity Analysis

Best Case Time Complexity: $O(1)$ [**When head = NULL
And, A node is
added to Empty
List**]

- *The best case occurs when the unrolled linked list is completely empty (**head == NULL**).*
- *The program simply allocates memory for a new head node, assigns the value to the first array index, and returns immediately without executing any loops.*

Worst Case Time Complexity: $O(N)$

- *The worst case occurs when the list already contains one or more nodes.*
- *The algorithm uses a while loop (**while** (**temp** $\rightarrow$ **next** $\neq$ **NULL**)) to traverse from the head node all the way to the last node in the sequence.*
- *Because it must visit every node to find the end, the time it takes scales linearly with **N**, where **N** is the total number of nodes in the list.*

Average Case Time Complexity: $O(N)$

The insertElement function does not ask for a position. By its fundamental design, it only appends to the very end of the list.

Because of this hardcoded behavior, there are no probabilities to calculate. The algorithm only has two possible physical states:

1. ***The list is empty:*** Takes $O(1)$ time.
2. ***The list has nodes:*** The program must traverse every single existing node to reach the end.

Hence, there is no real average case, $\therefore$ Worst Case = Average Case = $O(N)$

As every insertion into a populated list forces the program to traverse the entire list to find the end. Therefore, the average performance remains bound by the $O(N)$ traversal time.
